



ELSEVIER

European Journal of Operational Research 119 (1999) 314–325

EUROPEAN
JOURNAL
OF OPERATIONAL
RESEARCH

www.elsevier.com/locate/orms

Cluster based branching for the asymmetric traveling salesman problem

Jens Lysgaard *

Department of Management Science and Logistics, The Aarhus School of Business, Fuglesangs Alle 4, DK-8210 Aarhus V, Denmark

Abstract

This paper presents a new branching scheme for the asymmetric traveling salesman problem (ATSP) based on clusters. A cluster is defined as a node set with the characteristic that there exists an optimal solution in which the nodes in the node set are visited consecutively. The paper considers identification of clusters, implementation of a cluster based branching scheme, and cluster based dominance tests. The new approach is implemented in a branch and bound algorithm using a well-known additive bounding procedure. Considerable savings in computing time are obtained compared to previously published assignment based branch and bound algorithms for the ATSP. © 1999 Elsevier Science B.V. All rights reserved.

Keywords: Travelling salesman; Routing; Optimisation

1. Introduction

The asymmetric traveling salesman problem (ATSP) is the problem of finding a Hamiltonian circuit of minimum cost in a directed arc-weighted graph.

A characteristic of many branch and bound (BB) algorithms presented in the literature for the ATSP is that the linear assignment problem (AP) is applied for establishing lower bounds for the optimal solution to the ATSP. For example, in the algorithm by Carpaneto and Toth (1980) the AP bound is the exclusive lower bounding procedure.

A number of approaches have been presented for strengthening the bound compared to that obtained by the AP relaxation. Among additive bounding procedures (Fischetti and Toth, 1989) the AP relaxation is frequently used as the first of a series of bounding procedures. Important contributions along this line are, among others, the subtour contraction–compression routine by Christofides (1972), the restricted Lagrangean Approach by Balas and Christofides (1981), and the additive bounding procedure by Fischetti and Toth (1992).

As far as branching schemes are concerned, the two subtour elimination schemes by Bellmore and Malone (1971) and by Garfinkel (1973), respectively, dominate the literature on assignment-based branch and bound algorithms for the ATSP.

* Tel.: +45-86-15-55-88; fax: +45-86-15-19-43; e-mail: lys@hdc.hha.dk

Both schemes are based purely on the subtour configuration in the AP solution. While the identification of the branching to be used at any branch and bound node is computationally efficient in these branching schemes, their use also has the characteristic that any additional information on the problem structure that may be obtained by the bounding procedures is not utilized. This drawback of existing branching schemes is the origin of this paper.

In this paper we present a branching scheme in which additional information on the problem structure is utilized. The key concept of the new approach is that of a cluster, which is defined as a node set that can be visited consecutively in an optimal solution.

Regarding the overall composition of the resulting branch and bound algorithm, we generate the lower bounds by the additive bounding procedure by Fischetti and Toth (1992), as their algorithm is considered to be the most efficient among previously published assignment based branch and bound algorithms for the ATSP. The new algorithm presented in this paper may thus be considered as a modification of the Fischetti and Toth (1992) algorithm, the modification being that a standard subtour elimination scheme is replaced by a cluster based branching scheme.

2. Notation

Let there be given an ATSP instance specified by a complete graph $G = (N, A)$ with node set N (of which the nodes are numbered $1, 2, \dots, n$) and arc set A with associated arc costs $c_{ij} \forall (i, j) \in A$. With respect to the cost matrix we assume throughout that $c_{ii} = \infty \forall i \in N$ and that all costs are nonnegative. For notational simplicity, $x(S_1, S_2)$ denotes the expression $\sum_{i \in S_1} \sum_{j \in S_2} x_{ij}$ in the following, and $v(P)$ denotes the optimum objective value of any problem P .

The ATSP may be formulated as the following integer linear programming problem with variables x_{ij} , where $x_{ij} = 1$ if arc (i, j) is contained in the circuit, otherwise $x_{ij} = 0$:

$$\min \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}, \quad (1)$$

s.t.

$$\sum_{j=1}^n x_{ij} = 1 \quad \forall i \in N, \quad (2)$$

$$\sum_{i=1}^n x_{ij} = 1 \quad \forall j \in N, \quad (3)$$

$$x(S, N - S) \geq 1 \quad \forall S \subset N: 2 \leq |S| \leq n - 2, \quad (4)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A. \quad (5)$$

For a given ATSP instance, we define a cluster as a node set $K \subset N$ with $2 \leq |K| \leq n - 2$ for which there exists an optimal solution satisfying $x(K, N - K) = 1$.

As such, the nodes in a cluster may be visited consecutively in an optimal solution. It is noted that we are concerned with finding only one optimal solution; there may well exist alternative optimal solutions in which the nodes in a cluster are not visited consecutively.

The notation used in the following description is similar to the notation used in Fischetti and Toth (1992). Specifically, the problem r -SAAP denotes the *Shortest spanning r -antiarborescence problem*, which is the problem of finding n arcs of minimum total cost, such that (i) the outdegree of each node is exactly one, and (ii) node r can be reached from any other node. The degree-constrained problems r -SAADP and r -SADP are defined as follows. The problem r -SAADP is r -SAAP with the additional constraint that node r has indegree equal to one. The problem r -SADP is the problem of finding n arcs of minimum total cost, such that (i) the indegree of each node is exactly one, (ii) any node can be reached from node r , and (iii) the outdegree of node r is equal to one.

3. The algorithm

In this section the algorithm is outlined and subsequently described in more detail. The entire algorithm is a best-bound BB algorithm in which

the computations performed at each BB node can be outlined as follows.

Input: An $n \times n$ nonnegative cost matrix $C^{(0)} = (c_{ij}^{(0)})$ and a global upper bound GUB equal to the value of the best feasible solution found so far;

Step 1. Solve the AP problem on matrix $C^{(0)}$. Set $\delta := v(AP)$ and let $C^{(1)}$ be the resulting residual cost matrix. Determine a heuristic solution to the given instance, and let Z_H be the value of this heuristic solution. If $Z_H < GUB$, then set $GUB := Z_H$;

Step 2. Compute on matrix $C^{(1)}$ the cost L_j of the shortest path from node 1 to node $j \forall j \in N$. Set $c_{ij}^{(2)} := c_{ij}^{(1)} + L_i - L_j \forall i, j \in N$;

Step 3. Solve the 1-SAAP problem on matrix $C^{(2)}$. Let $C^{(3)}$ be the resulting residual cost matrix, and set $\delta := \delta + v(1 - SAAP)$;

Step 4. Set $C^{(4,0)} := C^{(3)}$;
for $r := 1$ **to** n **do**
begin
 Solve the r -SADP problem on matrix $C^{(4,r-1)}$. Let $C^{(4,r)}$ be the resulting residual cost matrix, and set $\delta := \delta + v(r - SADP)$;
end

Step 5. Set $C^{(5,0)} := C^{(4,n)}$;
for $r := 1$ **to** n **do**
begin
 Solve the r -SAADP problem on matrix $C^{(5,r-1)}$. Let $C^{(5,r)}$ be the resulting residual cost matrix, and set $\delta := \delta + v(r - SAADP)$;
end

Step 6. **for** $i, j := 1$ **to** n **do if** $c_{ij}^{(5,n)} \geq GUB - \delta$
then $c_{ij}^{(0)} := \infty$;

Step 7. Determine whether the AP solution in step 1 is still optimal on matrix $C^{(0)}$, given the added set of prohibited arcs in step 6. If this is not the case (i.e., if there among the n arcs in the AP solution in step 1 exists an arc (i, j) for which $c_{ij}^{(0)} = \infty$), repeat from step 1, otherwise go to step 8.

Step 8. Search for clusters on the basis of the information obtained during steps 1–6. If at least one cluster is identified, go to step 9, otherwise go to step 10.

Step 9. Perform cluster based dominance tests and determine the cluster based branches from the current BB node. Stop.

Step 10. Determine the branches from the current BB node based on a standard subtour elimination scheme extended with dominance tests. Stop.

As far as steps 1–6 are concerned, we have implemented the algorithm in accordance with the description in Fischetti and Toth (1992) except for the heuristic search in step 1. The heuristic solution is determined by means of a composite heuristic. First, a solution is determined by means of the patching algorithm (Karp, 1979), in which the AP subtours are patched in decreasing order of length, where length refers to the number of nodes in a subtour. The solution constructed by the patching algorithm is then improved, if possible, by the Or-opt string relocation procedure (Or, 1976), where strings of one, two and three nodes are considered. The identification of a strong upper bound is relatively important as it is involved in the identification of clusters, which motivates the search for a better heuristic solution in every node of the BB tree.

The test in step 7 is not included in the implementation by Fischetti and Toth (1992). The check for maintained optimality of the AP solution may result in a repetition of the whole procedure, as variables with zero residual costs after step 1 may have positive residual costs after step 6 due to the degree-constrained arborescence problems.

Steps 8 and 9 are the main contributions of this paper. The two steps are concerned with a branch and bound tree structure as illustrated in Fig. 1.

In relation to Fig. 1, assume that we are considering any node set $S \subset N$ with $2 \leq |S| \leq n - 2$ at problem P_0 . A disjunction of P_0 is obtained by the two constraints $x(S, N - S) = 1$ and $x(S, N - S) \geq 2$, respectively, corresponding to subproblems P_1 and P_2 . If a lower bound at P_2 can

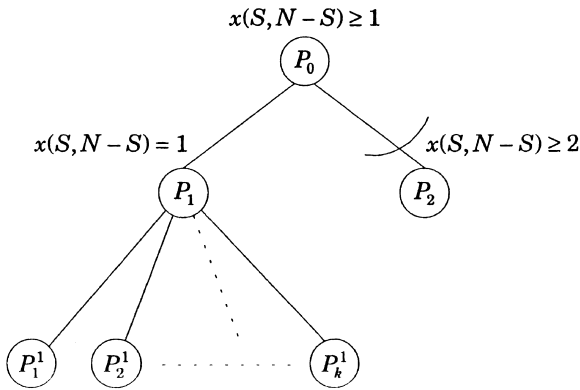


Fig. 1. Structure of branch and bound tree in the case of cluster branching.

be established such that this lower bound equals or exceeds the currently best feasible solution, then P_2 can clearly be discarded. In the alternative subproblem P_1 , we have that S is a cluster. The computations in step 8 search for an S for which the corresponding P_2 can be discarded immediately, i.e., without branching from P_2 . If this search is successful, then only P_1 , where S is a cluster, is considered.

If the search for a cluster is successful at P_0 , the computations in step 9 are concerned with generating the necessary branches from P_1 . An essential characteristic of P_1 is that any feasible solution contains a Hamiltonian path in S .

Assuming that $S = \{1, 2, \dots, k\}$, then the subproblems $P_1^1, P_2^1, \dots, P_k^1$ are considered as branches from P_1 . The branching is carried out in one of two possible ways, either an *entry* branching or an *exit* branching. In entry branching the subproblem P_i^1 is obtained by adding the constraint $x(N - S, \{i\}) = 1$ for $i = 1, 2, \dots, k$; similarly, the constraint $x(\{i\}, N - S) = 1$ is added for $i = 1, 2, \dots, k$ in the case of exit branching. Before deciding the type of branching, the subproblems of P_1 are analyzed by means of dominance tests, in which the number of necessary branches may be reduced compared to the original k branches. The chosen branching type (entry or exit branching) is the type of branching which generates the smallest number of subproblems. A key aspect of the branching scheme is that the cluster is contracted into a single node in each of the subproblems of P_1 .

The dominance tests in step 10 reuse some of the principles of step 9, as the standard subtour elimination scheme in fact involves a requirement that certain node sets are clusters.

In the following sections, we describe the computations involved in the cluster related aspects of the algorithm outlined in steps 8–10 in the above description.

3.1. Cluster identification

The algorithm involves two types of cluster identification. The first type is a relatively fast identification procedure based on the dual variables in the arborescence solutions. The second type, which is performed only if the first type does not provide at least one cluster, uses information from the arborescence solutions for providing candidate clusters, which then are tested in a computationally more expensive procedure.

3.1.1. Arborescence based identification

In steps 3–5, where at most $2n + 1$ arborescence problems are solved, we use the algorithm by Fischetti and Toth (1993). For each arborescence problem, a sequence ($k = 1, \dots, K$) of cuts $(S_k, N - S_k)$ associated with dual variables U_k are identified. For an entire arborescence problem, the transformation from original to residual costs is equivalent to performing the following steps for each identified cut associated with a positive dual variable:

$$\delta := \delta + U_k;$$

$$c_{ij} := c_{ij} - U_k \quad \forall (i, j) \in (S_k, N - S_k).$$

Let the cost matrix before and after this transformation be denoted by C^A and C^B , respectively. Then, for any feasible ATSP solution with p arcs in the cutset $(S_k, N - S_k)$, it is clear that $Z^A = Z^B + p \times U_k$, where Z^A and Z^B is the cost of the ATSP solution on matrix C^A and C^B , respectively. The transformation involved in the additive bounding procedure thus underestimates the lower bound by $(p - 1) \times U_k$.

Denoting the final lower bound from the additive bounding procedure by LB , it follows that

$LB + (p - 1) \times U_k$ is a valid lower bound for any feasible ATSP solution with p arcs in the cutset $(S_k, N - S_k)$. Correspondingly, if the current instance contains a better solution than GUB , then

$$LB + (p - 1) \times U_k < GUB. \quad (6)$$

For cutsets with $U_k \geq GUB - LB$ the only feasible solution to (6) is $p = 1$. In the search for a better solution than GUB we may thus discard the subproblem satisfying $x(S_k, N - S_k) \geq 2$, so that only the subproblem where $x(S_k, N - S_k) = 1$, equivalent to S_k being a cluster, needs to be considered.

During the bounding procedure, we store cutsets with positive dual variables. After the final step of the bounding procedure, clusters are found by identifying cutsets S_k satisfying $U_k \geq GUB - LB$. The final choice of the cluster to be used for branching is guided by two rules described in relation to the implementation of the algorithm (Section 4).

3.1.2. Identification based on artificial nodes

If the arborescence based identification procedure does not succeed in pointing out a cluster, the procedure described in this section is carried out.

When it is concluded that the arborescence based search does not identify a cluster, the list of stored cutsets $(S_k, N - S_k)$ and associated dual variables U_k is used for identifying *candidate clusters*, i.e., node sets for which the dual variables indicate that a more elaborate test could be worthwhile.

We use the rule that a cutset $(S_k, N - S_k)$ with dual variable U_k is a candidate cluster if $U_k \geq F \times (GUB - LB)$, where F is a prespecified parameter. The candidate clusters are considered in decreasing order of their associated dual variables. In the implementation we set a maximum on the number of considered candidate clusters.

The test for each candidate cluster is based on the SEAN (subtour elimination by artificial nodes) procedure (Lysgaard, 1996), which is a bounding procedure employing a transformation of the problem instance. The procedure adds artificial nodes to the problem instance for the purpose of improving the assignment based lower bound.

The transformation is illustrated in Fig. 2, where two nodes u and v are introduced to the instance. Each of the two nodes has ingoing arcs only from S_k and outgoing arcs only to $N - S_k$. Any feasible AP (and hence ATSP) solution thus contains at least two paths from S_k to $N - S_k$. The costs of the arcs to and from u and v are determined so that the optimal ATSP solution to the instance with u and v does not exceed the optimal ATSP solution on the original graph under the restriction $x(S_k, N - S_k) \geq 2$. The intention of the transformation is to obtain a lower bound equal to or exceeding GUB for the expanded graph, thus proving that the subproblem with $x(S_k, N - S_k) \geq 2$ can be fathomed immediately, so that S_k can be treated as a cluster.

Compared to the procedure in Lysgaard (1996), we employ a faster procedure for setting the costs

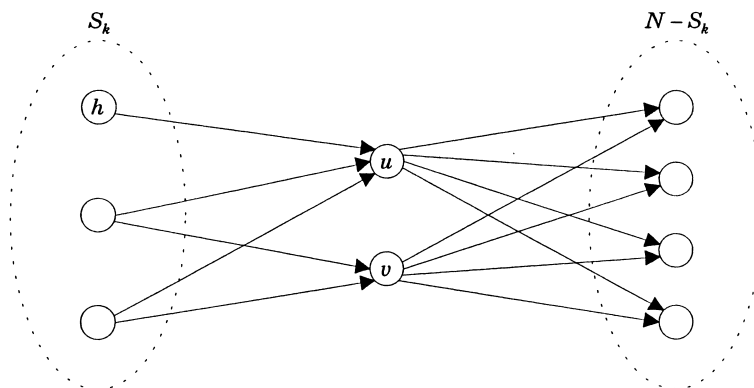


Fig. 2. Introduction of two artificial nodes for a cutset.

of the new arcs. The process in procedure FIRSTNODE takes the AP reduced costs from step 1 as input and determines the costs of arcs incident to node u .

Procedure FIRSTNODE

Comment $C = (c_{ij})$ are the AP reduced costs.

Step a. Set

$$c_{iu} := \min \{c_{ij} : j \in N - S_k\} \quad \forall i \in S_k;$$

Step b. Set

$$c_{uj} := \min \{c_{ij} - c_{iu} : i \in S_k\} \quad \forall j \in N - S_k.$$

Step c. Set

$$c_{ij} := \infty \quad \forall (i, j) \in (N - S_k, \{u\}) \cup (\{u\}, S_k).$$

The costs for the other node v are determined similarly except for one node $h \in S_k$, as described in procedure SECONDNODE. The purpose of identifying a node h is that the exclusion of the single arc (h, v) may provide a stronger lower bound of the expanded instance, as it may allow greater costs of the arcs incident to node v .

Procedure SECONDNODE

Step a. Choose $h \in S_k$ such that

$$c_{ih} = \min \{c_{iu} : i \in S_k\};$$

Step b. Set

$$c_{iv} := \min \{c_{ij} : j \in N - S_k\} \quad \forall i \in S_k - \{h\};$$

Step c. Set

$$c_{vj} := \min \{c_{ij} - c_{iv} : i \in S_k - \{h\}\} \quad \forall j \in N - S_k;$$

Step d. Set

$$c_{ij} := \infty \quad \forall (i, j) \in (N - S_k, \{v\}) \cup (\{v\}, S_k) \cup (h, v).$$

The obtained costs of the new arcs have the characteristic that for any arc $(i, j) \in (S_k, N - S_k)$, there exists the path (i, u, j) whose cost does not exceed c_{ij} . Similarly, for any arc $(i, j) \in (S_k - \{h\}, N - S_k)$, the cost of the path (i, v, j) does not exceed c_{ij} . Consequently, for any feasible ATSP solution satisfying $x(S_k, N - S_k) \geq 2$ on the original instance, there exists a feasible ATSP solution of no greater cost in the expanded instance.

For each candidate cluster, the transformation above is carried out and the resulting AP instance including the two artificial nodes is solved. The bounding steps 2–6 are then performed. If the lower bound equals or exceeds GUB , the node set is a cluster and the search stops.

If no cluster is identified after having considered all candidate clusters, we turn to identifying a subtour to be used for branching.

3.2. Choice of subtour for branching

We use a standard subtour branching scheme as the alternative to cluster branching, so that whenever no cluster is identified for branching, we choose a subtour to be eliminated by the Bellmore and Malone (1971) scheme. The subtour is chosen as described in the following.

The chosen subtour is always a subtour of minimum size, where the size of a subtour is the number of nodes contained in the subtour. The chosen subtour is rotated based on the penalty p_{ij} for each arc (i, j) contained in the subtour:

$$p_{ij} := \min \{\bar{c}_{ib} : b \neq j\} + \min \{\bar{c}_{aj} : a \neq i\},$$

where $\bar{c}_{ij}$ is the assignment reduced cost of x_{ij} . The penalty p_{ij} is a lower bound on the increase in the AP solution under the additional restriction $x_{ij} = 0$. We rotate the subtour so that the arc with the largest penalty will be prohibited in the first branch. In case of more than one subtour of minimum size the subtour containing the arc with the largest penalty is chosen.

3.3. Cluster branching

This section describes the computations involved in generating branches when a cluster has been identified and chosen as the basis for branching. Specifically, let node set S containing k nodes be the cluster chosen as the basis for branching.

For presentational convenience, we introduce some additional terms in relation to clusters.

Considering any cluster S , then for any path $[\dots, i, s, v_1, \dots, v_{k-2}, t, j, \dots]$ with $i, j \notin S$ and $S = \{s, v_1, \dots, v_{k-2}, t\}$, we say that i is a *departure node*, s is an *entry node*, t is an *exit node*, and j is an *arrival node*.

For any pair of entry node and exit node, the optimum path through the cluster can be computed independently from the remaining path in the ATSP solution. This forms the basis for the computations involved in cluster branching.

The first step is to compute the shortest Hamiltonian path in S for all pairs of entry nodes and exit nodes. These paths are computed as follows. Let d_{st} ($s, t \in S, s \neq t$) be the cost of the shortest Hamiltonian path in S from entry node s to exit node t . We compute $d_{st} \forall s, t \in S, s \neq t$ by means of the dynamic programming (DP) algorithm for the ATSP by Held and Karp (1962). By using this algorithm, we can solve the $k \times (k-1)$ shortest Hamiltonian path problems by solving only k ATSP instances each containing k nodes. For this we take each node in turn as the entry node s and execute the DP algorithm on this instance. Upon termination for each instance, the DP data structure contains d_{st} for all $t \in S - \{s\}$.

In the second step, we perform dominance tests in order to reduce the number of necessary branches. Without dominance tests, a branching with k branches can immediately be performed by taking each node in S in turn as the entry node or by taking each node in S as the exit node. The dominance tests try to reduce the number of entry nodes and exit nodes that necessarily must be considered. The principle is as follows.

For each pair of departure node i and arrival node j , the length λ_{ij} of the shortest path from i through S to j can be determined as follows:

$$\lambda_{ij} := \min \{c_{is} + d_{st} + c_{tj} : s, t \in S, s \neq t\} \\ \forall i, j \notin S, \quad i \neq j.$$

The dominance criterion is as follows. If, for all pairs of departure node i and arrival node j there exists a shortest path from i through S to j , on which a specific node s is not the entry node, then s needs not be considered any further as entry node. An equivalent argument applies to exit nodes.

In the implementation of dominance tests we use two matrices, D^+ and D^- , respectively, for computational efficiency. The D^+ entries are computed as follows:

$$D_{it}^+ := \min \{c_{iv} + d_{vt} : v \in S - \{t\}\} \\ \forall (i, t) \in (N - S, S).$$

As such, D_{it}^+ is the shortest path from departure node i to exit node t . Similarly, D_{sj}^- is defined as the shortest path from entry node s to arrival node j , i.e.:

$$D_{sj}^- := \min \{d_{sv} + c_{vj} : v \in S - \{s\}\} \\ \forall (s, j) \in (S, N - S).$$

The dominance tests involved in the possible reduction of exit nodes are as presented in procedure EXITS.

Procedure EXITS

Comment The set B^+ is the – possibly reduced – set of exit nodes.

```

 $B^+ := S;$ 
 $\lambda_{ij} := \min \{D_{it}^+ + c_{tj} : t \in B^+\} \quad \forall i, j \in N - S,$ 
 $i \neq j;$ 
repeat
   $T := \emptyset;$ 
  forall  $i, j \in N - S, \quad i \neq j$  do
    begin
      if ( $\lambda_{ij} < GUB$ ) and
        ( $\exists t \in B^+ : (\lambda_{ij} < D_{it}^+ + c_{tj} \quad \forall u \in B^+ - \{t\})$ )
      then  $T := T + \{t\};$ 
    end
  if  $B^+ - T \neq \emptyset$  then
    begin
      let  $u \in B^+ - T;$ 
       $B^+ := B^+ - \{u\};$ 
    end
  until  $B^+ - T = \emptyset$ 

```

It is noted that at most one exit node is eliminated per iteration in procedure EXITS. In each iteration, the set T contains those exit nodes that cannot be eliminated. This characteristic of a node t is determined as soon as a pair of departure node and arrival node is found for which t is the exit node on any shortest path through S . Upon ter-

mination, the set B^+ contains the nodes which necessarily must be considered as exit nodes.

A similar procedure is carried out in order to test the entry nodes. This procedure (which involves the D^- matrix instead of the D^+ matrix) terminates with a set B^- of nodes which necessarily must be considered as entry nodes.

The branching is then chosen based on B^+ and B^- . If $|B^+| < |B^-|$, we perform an *exit branching*, otherwise we perform an *entry branching*. In an exit branching, each of the nodes in B^+ is in turn taken as the required exit node, while in an entry branching, each of the nodes in B^- is in turn taken as the required entry node. The two types of branching are equivalent; the exit branching works as described in the following.

In each of the $|B^+|$ subproblems generated in the exit branching, the cluster is shrunk into a single node. Given that node $t \in B^+$ is the required exit node, new costs c_{it} from all departure nodes i to t are constructed as follows:

$$c_{it} := \min \{c_{iv} + d_{vt} : v \in S - \{t\}\} \quad \forall i \in N - S,$$

i.e., the constructed cost c_{it} is the length of the shortest Hamiltonian path from i through S to t . Shrinking the cluster implies that all nodes in $S - \{t\}$ are removed. The optimum solution to the new AP instance is found by the Hungarian algorithm in $|S| - 1$ iterations by taking in turn each node in $S - \{t\}$ and requiring it to be assigned to itself at zero cost.

The essential characteristic of the resulting AP solution is that the path $[i, t, j]$ represents, and equals in terms of costs, the optimum Hamiltonian path in the original instance from i through S to j given that t is the exit node of S .

3.4. Subtour reduction tests

Given that the subtour elimination scheme is used at the current BB node, we employ certain reduction tests before establishing the individual branch. For presentational convenience, we assume that the subtour chosen for branching consists of the arcs

$$((1, 2), (2, 3), \dots, (m-1, m), (m, 1)).$$

At branch k , the branching rule adds the following constraints to the instance (where indices are *mod* k):

$$x_{i,i+1} = 1: \quad i \in \{1, \dots, k-1\}, \quad x_{k,k+1} = 0.$$

At branch $k \geq 2$, the branching rule thus requires the path $[1, 2, \dots, k]$. As such, the node set $\{1, 2, \dots, k\}$ is required to be a cluster with node 1 as entry node and node k as exit node.

With a few modifications, the principles of the previously presented cluster dominance tests are reused as reduction tests in subtour branching. A main difference is that we are dealing with a new cluster at each branch in subtour branching.

At each branch $k \geq 2$, the reduction tests searches for additional valid constraints of the types $x_{u1} = 0$ and $x_{kv} = 0$ given the already considered branches $1, \dots, k-1$, i.e., the procedure tries to reduce the set of arrival nodes and departure nodes that must be considered.

The reduction tests differ between the cases $k = 2$ and $k \geq 3$, so we give distinct presentations of the two cases.

3.4.1. Reduction based on inversion

Given that we are considering the second branch, we compare the two paths $\mu_1 = [i, 2, 1, j]$ and $\mu_2 = [i, 1, 2, j]$ for all $i, j \notin \{1, 2\}$, $i \neq j$. As previously, we use the term departure node for i and arrival node for j given these path structures.

A path of type μ_2 , which is required at branch 2, is for all feasible paths compared to the corresponding feasible path μ_1 in branch 1. Letting λ_i denote the length of path μ_i , the argument in the reduction test is that the node pair (i, j) needs to be considered at branch 2 only if $\lambda_1 > \lambda_2$, otherwise a no worse solution for that node pair has already been considered at branch 1.

The reduction test simply performs an inversion of nodes 1 and 2 for each (i, j) . The steps are as shown in procedure INVERSION.

Procedure INVERSION

$I := \emptyset; J := \emptyset;$

forall $i, j \notin \{1, 2\}, i \neq j$ **do**

begin

$$\lambda_1 := c_{i2} + c_{21} + c_{1j};$$

$$\lambda_2 := c_{i1} + c_{12} + c_{2j};$$


```

if  $\lambda_1 > \lambda_2$  then
  begin
     $I := I + \{i\};$ 
     $J := J + \{j\};$ 
  end
end

```

At termination, the sets I and J contains those departure nodes and arrival nodes, respectively, that necessarily must be considered at branch 2. Given this information, we add the constraints $x_{i1} = 0 \forall i \notin I$ and $x_{2j} = 0 \forall j \notin J$ at branch 2. It is noted that if $I = \emptyset$ or $J = \emptyset$, then the second branch is completely dominated by the first branch.

3.4.2. Reduction based on dynamic programming

We consider a branch $k \geq 3$ and let $S = \{1, \dots, k\}$. The branching rule requires the path $[i, 1, \dots, k, j]$, i.e., S is required to be a cluster with node 1 as entry node and node k as exit node. The reduction test related to arrival nodes is described in the following. The test investigates the consequences of reordering the nodes in the required string so that the string has a different exit node.

We let $\mu_{tj} = [1, \dots, t, j]$ with $S = \{1, \dots, t\}$, i.e., μ_{tj} denotes a path with k arcs of which the first $k - 1$ arcs constitute a Hamiltonian path from 1 to t in S .

We say that any path μ_{tj} with $t \neq k$ dominates the path μ_{kj} if the length of μ_{tj} does not exceed the length of μ_{kj} . If there for a given path μ_{kj} exists a dominating path μ_{tj} , then μ_{tj} must be a feasible path at one of the branches $1, \dots, k - 1$, as μ_{tj} must deviate from $[1, \dots, k]$ at an earlier node on the path in order to end at node $t \neq k$. Hence the solution containing μ_{tj} has already been considered at an earlier branch, and we may thus disregard any solution containing μ_{kj} .

The test is carried out as follows. First we compute the length d_{1t} of the shortest Hamiltonian path in S from node 1 to node t for all $t \in S - \{1\}$. This is accomplished by the DP algorithm by solving only one ATSP instance given by S . Second, we test for each $j \notin S$ if μ_{kj} is dominated by at least one μ_{tj} , and if this is the case, then we add the constraint $x_{kj} = 0$ to the current instance.

A similar reduction procedure is carried out in relation to departure nodes. In this procedure, the concept of a dominating path relates to paths of the type $\mu_{is} = [i, s, \dots, k]$, where μ_{is} denotes a path with k arcs of which the last $k - 1$ arcs constitute a Hamiltonian path from s to k in S . The Hamiltonian path in S from all $s \in S - \{k\}$ to node k is found by the DP algorithm. For each $i \notin S$ we test if μ_{i1} is dominated by at least one path μ_{is} . If this is the case, then we add the constraint $x_{i1} = 0$ to the current instance.

3.5. Integration in branch and bound

In this section we briefly describe the integration in the BB structure of the computations introduced earlier in the sequel. The following outline highlights the integration of the branching decisions and the related dominance and reduction tests in the BB structure.

Stage 1. Let the original instance be the current (sub-)problem;

Stage 2. Compute lower and upper bound for the current (sub-)problem. This involves steps 1–7 in the previous outline;

Stage 3. Determine the branching to be performed from the current (sub-)problem. This involves searching for clusters, first on the basis of arborescence cuts, and, if this does not identify a cluster, then on the basis of artificial nodes. If a cluster is found, perform dominance tests and decide on entry or exit branching. Otherwise, identify a subtour as the basis for branching and perform the subtour reduction tests. This stage involves steps 8–10 in the previous outline;

Stage 4. Add the current (sub-)problem to the BB tree together with the information obtained in stage 3 on the generation of branches from this (sub-)problem;

Stage 5. Retrieve from the BB tree the (sub-)problem with the smallest lower bound. Generate the branches as prescribed by the branching information stored together with this (sub-)problem. For each generated subproblem, stages 2–4 are performed. Repeat stage 5 as long as the BB tree contains a subproblem with a lower bound smaller than the global upper bound.

It is noted that the branching to be used from a specific BB node is determined already when that node is generated in the tree. In the case of cluster branching, the dominance tests are also performed during the generation of that node, and, similarly, the subtour reduction tests are also performed when generating the BB node in the case of subtour branching. It is necessary to keep the information from dominance and reduction tests in the BB tree, as the retrieval of the BB node involves generating that node by following the path from the root of the BB tree. Alternative implementations involving backtracking in the BB tree have been avoided in order to avoid implementing the reverse of the shrinking process performed during branching.

4. Implementation

The branch and bound algorithm has been tested on the problem instances in TSPLIB (Reinelt, 1991). Some of the instances were dis-

carded during preliminary tests as they were solved at the root node (i.e., without branching). This applies to the 17-nodes instance ‘BR17’ and to the four largest problems ‘RBG323’, ‘RBG358’, ‘RBG403’, and ‘RBG443’, respectively. These four large instances were remarkably easily solved; the AP lower bound and the upper bound from the patching algorithm were equal in all four cases at the root node, so that the optimum was found immediately in all four cases. Among the remaining instances we have found during all tests that the instance ‘FTV170’ could not be solved due to excessive memory requirements. This instance has therefore also been excluded.

4.1. Parameters

The algorithm involves a number of parameters. We have performed a number of runs of the algorithm on the entire set of test problems in order to determine those parameter values yielding the best overall performance. Table 1 contains

Table 1
Computational results

Name	<i>n</i>	Subtour branching		Cluster branching					Time ratio
		Time	BB nodes	Time	BB nodes	Cluster nodes	Subtour nodes	Cluster ratio	
FT53	53	1.23	199	0.30	39	5	11	0.31	0.24
FT70	70	1.75	158	0.41	60	5	12	0.29	0.23
RY48P	48	17.9	3660	15.2	1695	474	301	0.61	0.85
KRO124P	100	510.8	17181	267.2	6232	2050	1114	0.65	0.52
P43X2	43	>3600	>98680	7.0	6480	312	1641	0.16	<0.002
FTV33	34	0.25	76	0.28	59	11	29	0.28	1.12
FTV35	36	1.31	757	0.57	273	29	63	0.32	0.44
FTV38	39	1.57	987	0.82	388	15	77	0.16	0.52
FTV44	45	0.48	224	0.39	152	7	37	0.16	0.81
FTV47	48	4.2	2016	2.8	1074	22	252	0.08	0.67
FTV55	56	13.5	7097	5.2	1916	74	257	0.22	0.39
FTV64	65	15.2	4841	11.2	3186	41	462	0.08	0.74
FTV70	71	100.5	21468	96.4	15617	502	3441	0.13	0.96
FTV90	91	22.0	1801	10.8	666	34	239	0.12	0.49
FTV100	101	453.9	26480	53.0	2594	203	935	0.18	0.12
FTV110	111	936.8	52173	211.9	9010	449	3276	0.12	0.23
FTV120	121	3538.5	125989	1235.5	39780	1333	13872	0.09	0.35
FTV130	131	416.2	12597	230.5	5870	270	2524	0.10	0.55
FTV140	141	707.8	19425	328.4	6633	531	2553	0.17	0.46
FTV150	151	361.6	8359	202.6	3502	293	1439	0.17	0.56
FTV160	161	>3330	>30445	2389.6	49915	1763	15867	0.10	<0.67

results only for this chosen combination of parameter values, which are as follows.

- In the arborescence based cluster identification, we store only those cutsets for which the corresponding cluster does not contain more than 8 nodes. This maximum also applies to candidate clusters, as only the stored cutsets are considered in the identification of candidate clusters;
- If more than one cluster is identified in the arborescence based identification, we choose the cluster covering the largest number of subtours; we say that a subtour is covered by a cluster if it has at least one node in common with the cluster. In case of ties we prefer the cluster with the smallest number of nodes;
- In the selection of candidate clusters we use the value $F = 0.3$ in the inequality $U_k \geq F \times (GUB - LB)$;
- If more than 5 candidate clusters satisfy the requirement $U_k \geq F \times (GUB - LB)$, we select the 5 candidate clusters with the largest dual variables;
- The subtour reduction tests are performed only up to the first 10 branches from each branch and bound node. This rule is used for the purpose of limiting the computing time of the DP algorithm in the reduction tests.

We use the penalty computations and subtour rotation in both algorithms, i.e., in a traditional subtour elimination scheme as well as in the cluster based algorithm.

It is noted that the cluster branching scheme in any case is tractable only for relatively small clusters (e.g., clusters containing up to 16 nodes), as the time as well as space requirement of the DP algorithm more than doubles for each additional node in the instance (Held and Karp, 1962).

4.2. Computational results

Table 1 summarizes the results of the cluster based algorithm with the described parameter settings as well as the results with a traditional subtour elimination scheme, where the latter is represented by the heading ‘subtour branching’ in Table 1. The two columns with the heading ‘BB

nodes’ contain the total number of nodes generated in the branch and bound tree. As far as cluster branching is concerned, Table 1 also contains the columns ‘cluster nodes’, ‘subtour nodes’, and ‘cluster ratio’. The ‘cluster nodes’ column contains the number of branch and bound nodes at which a cluster branching is determined. Similarly, the ‘subtour nodes’ column contains the number of branch and bound nodes at which a standard subtour branching is determined. The ‘cluster ratio’ is the ratio of ‘cluster nodes’ to the sum of ‘cluster nodes’ and ‘subtour nodes’. For example, for the instance ‘FT53’, a total of 39 nodes is generated in the branch and bound tree, and among these, 16 nodes are kept in the tree after being generated (the remaining 23 nodes being fathomed), and a cluster branching is determined at 5 of the 16 nodes. The ‘time ratio’ is the computing time with cluster branching relative to the computing time with subtour branching. The computing times are given in CPU seconds on a DEC Alpha 2100 5/250.

The most remarkable single result is the performance of the cluster branching scheme on the instance ‘P43X2’. This instance is well known to be extremely hard when solved by subtour branching, e.g., Balas et al. (1993) report that ‘P43X2’ was not solved within 32 CPU h on a SUN Sparcstation 330 with subtour branching. The excessive computing time was confirmed in our test, where we terminated the process with subtour branching after one CPU hour. In contrast, the instance is solved with cluster branching in seven CPU s.

The extreme effect on the ‘P43X2’ instance is a consequence of the fact that the clusters and subtours chosen for branching in ‘P43X2’ in many cases consist of *identical* nodes; we call two nodes identical if they can be interchanged in any feasible ATSP solution without changing the objective value. When a cluster consists of identical nodes, the cluster dominance tests effectively reduces the necessary branches to only one branch. Similarly, the reductions based on inversion and dynamic programming for subtour branching has the same effect when applied to subtours consisting of identical nodes.

Concerning the overall picture, the performance of cluster branching is consistently better

than subtour branching (with the only exception being the ‘FTV33’ case). In addition, the relative saving in computing time is of a considerable magnitude in several cases. It is also noted that cluster branching consistently generates a smaller number of branch and bound nodes than subtour branching.

Regarding the ‘cluster ratio’ values, which may be taken as performance indices of the cluster identification procedures, the tests show that the search for a cluster is successful in 10%–30% of the branch and bound nodes on the majority of problem instances.

5. Conclusions and perspectives

This paper has introduced the concept of cluster branching in relation to the ATSP. The obtained computational results show that the cluster branching approach performs consistently better than a conventional subtour elimination scheme. Cluster branching has the potential of significantly reducing the solution time compared to a branching scheme based purely on subtour elimination.

The new branching scheme does, however, fit into a much broader class of problem types and algorithmic contexts than represented by the current paper. As far as the algorithmic context is concerned, we find it important to emphasize that the requirement that a subproblem generated by a constraint of the type $x(S, N - S) \geq 2$ can be immediately fathomed, as illustrated in Fig. 1, is a consequence of the fact that the lower bounding procedures are all network algorithms in which such an additional constraint is not easily handled. In contrast, if a linear programming relaxation is used, then this constraint can easily be handled, i.e., the disjunction $x(S, N - S) = 1 \vee x(S, N - S) \geq 2$ can be used generally in that context. Furthermore, with respect to a broader class of problem types, it is noted that cluster branching

has a structure which is of potential interest to sequencing problems in general. As such, a broad spectrum of further analyses is open for future research.

References

- Balas, E., Christofides, N., 1981. A restricted Lagrangean approach to the traveling salesman problem. *Mathematical Programming* 21, 19–46.
- Balas, E., Ceria, S., Cornuéjols, G., 1993. A lift-and-project cutting plane algorithm for mixed 0-1 programs. *Mathematical Programming* 58, 295–324.
- Bellmore, M., Malone, J.C., 1971. Pathology of traveling-salesman subtour-elimination algorithms. *Operations Research* 19, 278–307.
- Carpaneto, G., Toth, P., 1980. Some new branching and bounding criteria for the asymmetric travelling salesman problem. *Management Science* 26, 736–743.
- Christofides, N., 1972. Bounds for the travelling-salesman problem. *Operations Research* 20, 1044–1056.
- Fischetti, M., Toth, P., 1989. An additive bounding procedure for combinatorial optimization problems. *Operations Research* 37, 319–328.
- Fischetti, M., Toth, P., 1992. An additive bounding procedure for the asymmetric traveling salesman problem. *Mathematical Programming* 53, 173–197.
- Fischetti, M., Toth, P., 1993. An efficient algorithm for the minimum arborescence problem on complete digraphs. *ORSA Journal of Computing* 5, 426–434.
- Garfinkel, R.S., 1973. On partitioning the feasible set in a branch-and-bound algorithm for the asymmetric traveling-salesman problem. *Operations Research* 21, 340–343.
- Held, M., Karp, R.M., 1962. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics* 10, 196–210.
- Karp, R.M., 1979. A patching algorithm for the nonsymmetric traveling-salesman problem. *SIAM Journal of Computing* 8, 561–573.
- Lysgaard, J., 1996. Subtour elimination by artificial nodes for the asymmetric traveling salesman problem, Working paper B 96-1, Department of Management Science and Logistics, The Aarhus School of Business.
- Or, I., 1976. Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking, Ph.D. thesis, Northwestern University, Evanston, IL.
- Reinelt, G., 1991. TSPLIB – A Traveling Salesman Problem Library. *ORSA Journal of Computing* 3, 376–384.